

Processes

Luís Lopes

Faculdade de Ciências
Universidade do Porto

`lmlopes@fc.up.pt`

Overview

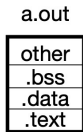
- ① what is a process
- ② address space
- ③ process control block
- ④ life-cycle
- ⑤ process queues
- ⑥ creation and synchronization

Programs and Processes

- *process = running instance of a program*
- one program may give rise to multiple processes
- ... but not the other way around
- operating system assigns to each starting process:
- ... a unique integer identifier or Process ID (PID)
- ... a (virtual) address space
- (holds the code and runtime data)
- ... a kernel data structure called Process Control Block (PCB)
- (holds the state of the process)

Address Space and PCB

compilation
generates
executable
program



program
execution
starts
process

\$./a.out
→



operating system
assigns address
space to process

+

Process
Control
Block
(PCB)

a kernel
data structure
that keeps
track of
process state

Address Space

- the maximum size of address space limited by
- ... number of addressing bits of the architecture
- e.g., 32 bit $\implies 2^{32} = 2^2 \times 2^{10} \times 2^{10} \times 2^{10}$ bytes = 4 GB
- e.g., 64 bit $\implies 2^{64}$ bytes = 16 EB
- however, current 64 bit architectures only use
- 48 bits $\implies 2^{48}$ bytes = 256 TB
- 52 bits $\implies 2^{52}$ bytes = 4 PB

(more on this latter)

Address Space

- static space: `.text`, `.data` and `.bss`
- (`.bss` = block starting symbol, uninitialized global variables)
- stores the binary code for the program and static data
- ... as copied from the program executable binary file
- dynamic space: `stack` and `heap`
- this part of the address space is used only during runtime
- stack keeps track of function calls
- data stored in the stack is ephemeral
- heap used to store longer-lived data
- variables in the stack keep pointers to heap data

Address Space

```

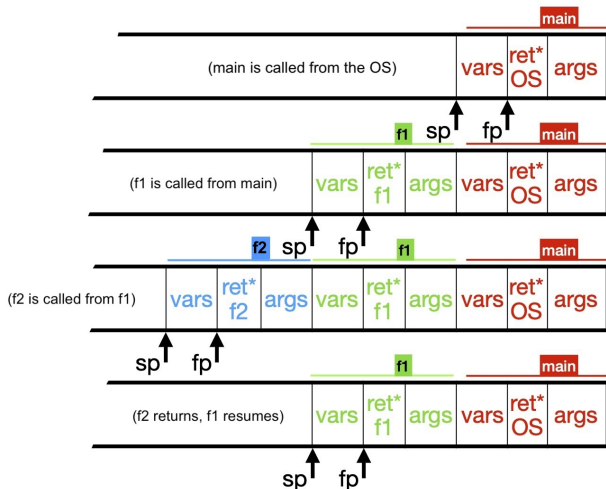
int main() {
    main vars;
    f1(f1 args);
    return;
}

void f1(f1 args) {
    f1 vars;
    f2(f2 args);
    return;
}

void f2(f2 args) {
    f2 vars;
    ...
    return;
}
    
```

ret* f1 →

ret* f2 →



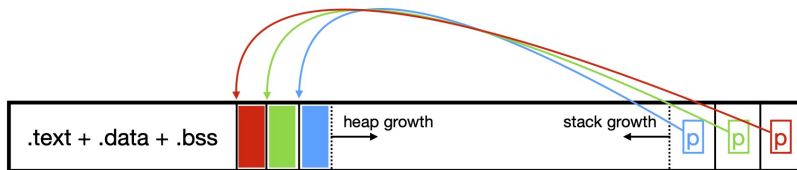
Address Space

- heap management done by the programmer:
- `void* malloc(size_t size)`
- `void* calloc(size_t count, size_t size)`
- `void* realloc(void *ptr, size_t size)`
- `void free(void *ptr)`
- the functions are available from `libc`
- C keeps information on allocated blocks
- is the block used? what is the size of the block?
- freed blocks are reused in further `malloc()` calls
- blocks have different sizes, hence fragmentation of heap

Address Space

- heap size is limited
- hence, at some point, `malloc()` may fail
- operating system may extend heap size up to a limit
- usually through system calls such as:
- `void* brk(const void* addr)` (mostly deprecated)
- `void* mmap(void* addr, ...)` (current implementations)
- the operating system limits heap growth, c.f. `getrlimit()`

Process Address Space



```
int main() {  
    char* p = (char*)malloc(16*sizeof(char));  
    f1();  
    return 0;  
}
```

```
void f1() {  
    int* p = (int*)malloc(4*sizeof(int));  
    f2();  
    return;  
}
```

what happens
to space in heap
when functions
return?

```
void f2() {  
    double* p = (double*)malloc(2*sizeof(double));  
    ...  
    return;  
}
```

why (in this
example) are the
blocks of the
same size?

Process Control Block

- a large kernel data structure (`struct`)
- keeps information on execution progress
- e.g.,
- PID, status, parent process, child processes,
- CPU registers (including `$pc`, `$sp`, `$fp`, ...)
- map of memory areas being used by process
- list of files, pipes, fifos, sockets, ..., opened by process
- list of signal handlers, pending signals
- scheduling information
- resource limits

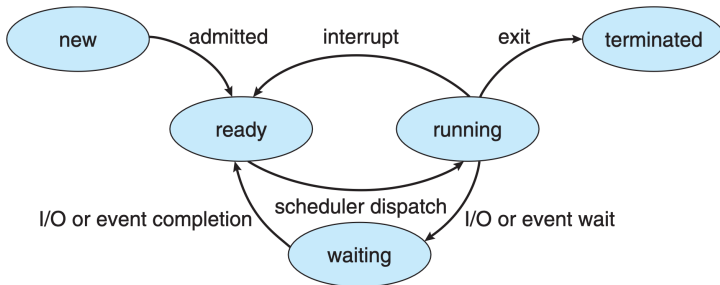
Process Control Block

e.g., in the Linux kernel ...

```
struct task_struct {
    volatile long state; // RUNNING, STOPPED, ...
    struct thread_info *thread_info; // thread info
    struct mm_struct *mm; // address space info
    struct fs_struct *fs; // filesystem info
    struct files_struct *files; // file descriptors
    struct signal_struct *signal; // signal info
    struct sighand_struct *sighand; // signal info
    ...
    /* various other fields */
    ...
};
```

Process Life-Cycle

- the life-cycle of a process is depicted in the following diagram:

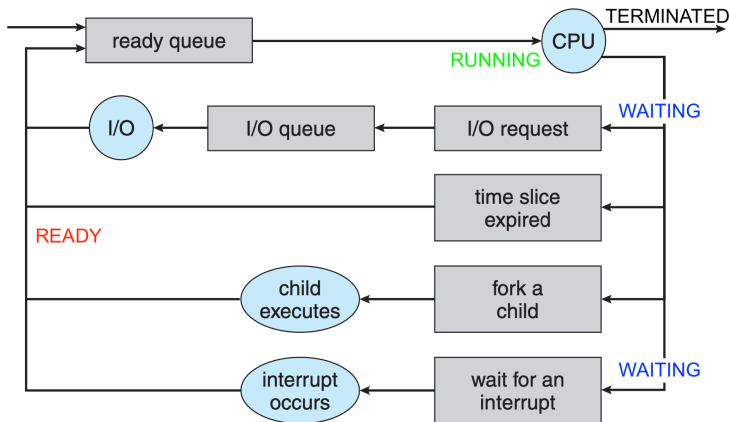


Process Life-Cycle

- a new process p (just created) is READY to be executed
- it awaits for the operating system to assign it the CPU
- when that happens p transitions to state RUNNING
- during its execution p may ask for an I/O operation
- (e.g., read data from a file, waiting for an interrupt)
- the operating system suspends p and its state is WAITING
- eventually, the I/O operation completes and p is READY again
- p will do READY - RUNNING - WAITING - READY - ... loop
- ... several times until it ends and is in state TERMINATED
- all the resources it used are reclaimed by the operating system

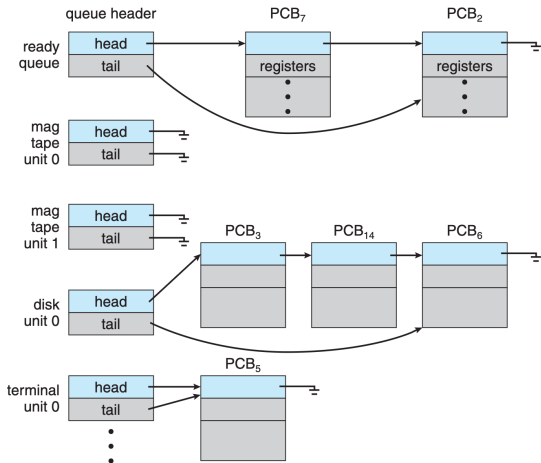
Process Life-Cycle

- the following diagram shows events that trigger state changes:



Process Life-Cycle

- the operating system handles these state changes by
- ... moving the process PCBs between queues in kernel space



Programmatic Process Management

- in UNIX systems the kernel resorts
- ... to several system calls to create, execute,
- ... synchronize and terminate processes
- the most important ones are:
- `fork()` $\implies$ create a copy of current process
- `exec()` $\implies$ change the program of current process
- `wait()` $\implies$ wait for the termination of child process
- `exit()` $\implies$ terminate the execution of current process
- some of these have variants,
- e.g., `execlp()` and `execvp()`
- e.g., `waitpid()`

Programmatic Process Management

⇒ `fork()` system call

- creates copy of current process
- this copy is called a child process
- the original is the parent
- thus, one process gives rise to two
- copy differs only in the PID

Programmatic Process Management

⇒ `fork()` system call

- the call returns to **both processes**
- the parent receives the PID of the child
- the child receives 0
- this asymmetry is used to make parent
- ... and child do different things
- but, this is not very flexible
- it does not allow child to execute any process

Programmatic Process Management

⇒ `execvp()` / `execvp()` system calls

- initialize the current process with a new program
- the program is given as argument
- the new program starts from beginning
- the original program is obliterated and its data structures reset
- but the process maintains its address space, PCB and PID

Programmatic Process Management

⇒ `execlp()` / `execvp()` system calls

- `exec()` overwrites the `.text`, `.data`, `.bss`
- resets the heap and stack
- loads the `main()` arguments
- starts the (new) program
- `fork() + exec()` can make child execute any process

Programmatic Process Management

⇒ `wait()` / `waitpid()` system calls

- used by parent process to wait for terminating child processes
- this should (almost) always be done
- we can recover the exit status of the child process (more later)
- also, if child process exits without parent knowledge
- parent cannot report their demise to operating system
- operating system cannot release resources allocated to them
- e.g., memory pages / segments, I/O buffers, ...
- child processes remain in system process table for some time
- such processes are called *zombie processes*
- they are eventually collected but zombies are to be avoided

Programmatic Process Management

⇒ `wait()` / `waitpid()` system calls

- sometimes it is desirable that parent process exits before child
- therefore not calling `wait()` on its children
- in this situation, child processes may continue to run
- they are called *orphan processes*
- they are adopted by the `init` system process
- operating system service *daemons* are created in this way

Programmatic Process Management

⇒ `exit()` system call

- can at first seem like a new name for `return`
- but `exit` terminates the process anywhere in the code
- `return` only results in termination if from `main()`
- `exit` performs cleanup before returning control to OS
- allows the OS to reclaim system resources allocated to process

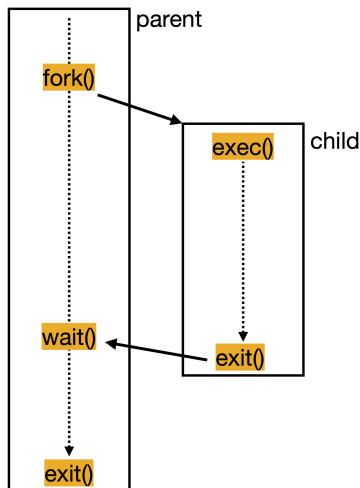
Programmatic Process Management

⇒ `exit()` system call

- the argument of the call indicates process exit status
- in UNIX-like systems, a value of 0 means no errors
- other positive integer values signal different errors
- if the parent calls `wait()` it can recover this info
- we will use `EXIT_SUCCESS` and `EXIT_FAILURE`
- these are macros with values of 0 and 1, respectively

Process Life-Cycle

- these system calls work in tandem as in this typical flow:



Programmatic Process Management

⇒ 2 processes

```
int main(int argc, char* argv[]) {  
    fork();  
    exit(EXIT_SUCCESS);  
}
```

⇒ 4 processes

```
int main(int argc, char* argv[]) {  
    fork(); fork();  
    exit(EXIT_SUCCESS);  
}
```

⇒ 8 processes

```
int main(int argc, char* argv[]) {  
    fork(); fork(); fork();  
    exit(EXIT_SUCCESS);  
}
```

Programmatic Process Management

⇒ 2^n processes

```
int main(int argc, char* argv[]) {  
    // n fork() calls  
    fork(); fork(); fork(); ... ; fork();  
    exit(EXIT_SUCCESS);  
}
```

⇒ same as above, n given as argument

```
int main(int argc, char* argv[]) {  
    for (int i = 0; i < atoi(argv[1]); i++) fork();  
    exit(EXIT_SUCCESS);  
}
```

Programmatic Process Management

⇒ full form of fork - wait pattern

```
int main(int argc, char* argv[]) {
    pid_t pid = fork();
    if (pid == -1) exit(EXIT_FAILURE);
    if (pid == 0) {
        printf("CHILD: PID = %d\n", getpid());
        exit(EXIT_SUCCESS);
    }
    else {
        int retval = wait(NULL);
        if (retval == -1) exit(EXIT_FAILURE);
        printf("PARENT: PID = %d\n", getpid());
        exit(EXIT_SUCCESS);
    }
}
```

Programmatic Process Management

`nop1.c` $\Rightarrow$ run command `argv[1]` without arguments

```
int main(int argc, char* argv[]) {
    pid_t pid = fork();
    if (pid == -1) exit(EXIT_FAILURE);
    if (pid == 0) {
        int retval = execlp(argv[1], argv[1], NULL);
        if (retval == -1) exit(EXIT_FAILURE);
    }
    else {
        int retval = waitpid(pid, NULL, 0);
        if (retval == -1) exit(EXIT_FAILURE);
    }
    exit(EXIT_SUCCESS);
}
```

Programmatic Process Management

`nop2.c` $\Rightarrow$ run command `argv[1]` including arguments

```
int main( int argc, char* argv[] ) {  
    execvp(argv[1],argv+1);  
    exit(EXIT_FAILURE);  
}
```

e.g.:

```
$ gcc -Wall nop1.c -o nop1  
$ ./nop1 wc -l -c -w nop1.c  
( runs 'wc' only )  
$ gcc -Wall nop2.c -o nop2  
$ ./nop2 wc -l -c -w nop2.c  
( runs 'wc -l -c -w nop2.c' )
```

Programmatic Process Management

bsh.c $\Rightarrow$ basic shell runs multiple commands no arguments

```
int main(int argc, char* argv[]) {
    for( ; ; ) {
        printf("$ ");
        char* cmd = read_command(stdin);
        pid_t pid = fork();
        if(pid == -1) exit(EXIT_FAILURE);
        if(pid == 0) {
            int retval = execlp(cmd, cmd, (char *)NULL);
            if(retval == -1) exit(EXIT_FAILURE);
        }
        int retval = waitpid(pid, NULL, 0);
        if (retval == -1) exit(EXIT_FAILURE);
    }
    exit(EXIT_SUCCESS);
}
```